

Ejercicios de Agentes Inteligentes

Curso introductorio de Inteligencia Artificial — Simulador de forrajeo ePuck (Russell & Norvig)

Sobre este documento

Este documento crece a lo largo del curso: cada nueva actividad se agrega en el orden en que se cubre el tema correspondiente en clase (agente table-driven, reflejo simple, reflejo basado en modelo, basado en metas, basado en utilidad, y agente que aprende).

Convención bilingüe: los enunciados están en español; los nombres de archivos, funciones, variables y conceptos técnicos del código se dejan en inglés (por ejemplo, `table_size()`, `bins_per_sensor`, `percept sequence`), ya que así están escritos en el proyecto y en la bibliografía del curso.

Cada actividad tiene dos partes:

- Para el alumno: objetivo, contexto, tarea y entregable.
- Manual del profesor: respuesta esperada, errores comunes, rúbrica y variantes — pensado para que cualquier ayudante de clase pueda calificar sin haber diseñado el ejercicio.

Actividad 1: Agente Table-Driven

Objetivo de aprendizaje

Entender por qué un agente table-driven es una arquitectura válida en teoría (mapea cada percept, o secuencia de percepts, a una acción mediante una tabla) pero se vuelve inviable en la práctica en cuanto la percepción deja de ser trivialmente pequeña — y ser capaz de argumentarlo con números, no solo de memoria.

Contexto

Archivos que vas a necesitar (carpeta training_wheels/):

- agents/table_driven_agent.py — contiene la función `table_size()`, que calcula cuántas filas necesitaría la tabla de un agente table-driven para el robot forrajero, dada una discretización de sus sensores.
- peas_foraging.json — para recordar cuántos sensores tiene el robot y qué otras variables percibe (food bearing, odor).

Tarea

Este ejercicio es semi-abierto: se te da el objetivo final, tú decides cómo llegar (qué corridas hacer, qué supuestos tomar, cómo presentar el argumento). Debes producir un reporte corto que responda lo siguiente:

1. Calcula el tamaño de la tabla (número de filas) para al menos 5 niveles de discretización distintos de los sensores de proximidad (por ejemplo, desde muy grueso hasta razonablemente fino). Usa o modifica `table_size()` según lo necesites.
2. Convierte esos tamaños a un estimado de memoria real (bytes, luego a la unidad que tenga sentido: KB, MB, GB, TB...), siendo explícito sobre el supuesto que usaste para el tamaño de cada fila (por ejemplo, cuántos bytes hacen falta para guardar una acción entre 5 posibles).
3. Compara al menos dos de tus resultados contra una referencia real y concreta (por ejemplo: la RAM de una laptop típica, el almacenamiento de un teléfono, la capacidad de un disco duro común). Busca esos números tú mismo/a.
4. Russell & Norvig definen al agente table-driven sobre la SECUENCIA de percepts vistos hasta el momento, no solo el percept actual. Argumenta (con números si puedes, o al menos con la fórmula correcta) qué le pasa al tamaño de la tabla si el agente necesitara recordar, digamos, los últimos 3 a 5 pasos en vez de solo el instante actual.
5. Escribe una conclusión de máximo un párrafo: ¿en qué nivel de discretización, y por qué razón concreta (no solo "se hace grande"), dirías que el enfoque table-driven deja de ser viable para este robot?

Entregable

Un reporte corto (puede ser un documento, un notebook, o una combinación de ambos) con: la tabla de resultados del punto 1, el cálculo de memoria del punto 2, la comparación del punto 3, el análisis del punto 4, y la conclusión del punto 5. No se espera código adicional más allá de, en su caso, adaptar `table_size()`.

Lista de autoevaluación

- ¿Mostré al menos 5 niveles de discretización, no solo los 3 que ya vienen de ejemplo en el archivo?
- ¿Convertí explícitamente de bytes a una unidad legible, mostrando la cuenta (no solo el resultado final)?
- ¿La referencia real que usé para comparar tiene una fuente (aunque sea una búsqueda rápida), no es un número inventado?
- ¿Distinguí claramente entre "tabla sobre el percept actual" y "tabla sobre la secuencia de percepts", y no las mezclé?
- ¿Mi conclusión da una razón concreta (memoria, tiempo de construcción, calidad de control) y no solo "es mucho"?

Manual del profesor — Actividad 1

Esta sección es para quien imparte o califica el curso; no se comparte con el alumnado antes de la entrega.

Respuesta esperada

Con la función `table_size()` tal cual viene (8 sensores, 2 variables extra con 4 bins cada una), los valores de referencia son:

bins_per_sensor	Filas de la tabla	Memoria (1 byte/fila)
3 (muy grueso)	104,976	~102 KB
5 (grueso)	6,250,000	~6 MB
10 (moderado)	1,600,000,000	~1.6 GB
20 (fino)	419,430,400,000 aprox.	~419 GB
50 (muy fino)	$\sim 2.56 \times 10^{15}$	~2.56 PB

Trampa pedagógica a vigilar

Muchos equipos se van a quedar satisfechos con el resultado de 10 bins (~1.6 GB) y van a concluir que "sí cabe en una laptop, entonces sí es viable". Ese es exactamente el punto a corregir, por dos razones que conviene sacar en la discusión grupal, no dárselas de entrada:

- 1.6 GB de granularidad de 10 buckets por sensor es una discretización demasiado tosca para un control suave — el robot oscilaría entre acciones al cruzar los límites de los buckets. Para un control razonable hacen falta muchos más niveles por sensor, y ahí el tamaño se dispara (columna de 20 y 50 bins arriba).
- Más importante: la tabla de la función `table_size()` es solo sobre el percept ACTUAL. La definición real de Russell & Norvig de un agente table-driven es sobre la secuencia completa de percepts. Si el agente necesitara memoria de los últimos k pasos, el tamaño correcto es $(\text{tamaño_de_un_percept})^k$, no una simple multiplicación adicional. Con el valor ya "aceptable" de 3 bins (104,976 percepts posibles) y $k=3$ pasos de memoria, el tamaño ya es $104,976^3 \approx 1.16 \times 10^{15}$ — es decir, el argumento de "cabe en una laptop" se cae en cuanto se agrega cualquier noción de memoria, incluso con la discretización más generosa posible.

Si algún equipo no llega solo a este segundo punto, es la pregunta correcta para hacerles en la revisión antes de darles la respuesta.

Rúbrica de evaluación (10 pts)

Criterio	Puntos
Calculó correctamente al menos 5 niveles de discretización (no solo los 3 de ejemplo)	2
Convirtió a memoria real con supuesto explícito y correcto (bytes por fila)	2
Comparó contra al menos una referencia real de hardware, con fuente	2
Identificó y calculó correctamente el efecto de la secuencia de percepts (no solo el percept actual)	2
Conclusión con razón concreta (no genérica) y bien argumentada	2

Variante si sobra tiempo

Pregunta tipo Fermi para cerrar la discusión en clase: "si pudieran generar y guardar una fila de la tabla por nanosegundo, ¿cuánto tiempo tomaría construir la tabla completa para el nivel más fino que calcularon?" — para la fila de 50 bins ($\sim 2.56 \times 10^{15}$), a un nanosegundo por fila son aproximadamente

30 días de cómputo ininterrumpido SOLO para enumerar las filas, sin siquiera decidir qué acción poner en cada una. Es un buen cierre visual para la idea de "inviable en la práctica".

Extensión opcional: pedirles que relacionen esta discusión con el grid de navegación de 10-15 nodos que se usará más adelante para búsqueda y para el agente que aprende (Q-learning) — ¿por qué esa tabla sí es viable, si también es una tabla? (Respuesta esperada: porque no se indexa sobre el percept crudo, sino sobre una abstracción del mundo diseñada a propósito para ser pequeña.)

Actividad 2: Reflejo Simple vs. Reflejo Basado en Modelo

Objetivo de aprendizaje

Entender que una arquitectura "más sofisticada" (con estado interno) no garantiza mejor desempeño si ese estado interno depende de una suposición que ya no se cumple en el contexto donde se usa — y practicar diagnóstico empírico de código ajeno antes de intentar arreglarlo.

Contexto

Archivos que vas a necesitar (carpeta training_wheels/):

- demo_level0_reflex.py — para correr ambos agentes.
- agents/simple_reflex_agent.py y agents/model_based_reflex_agent.py — los dos agentes a comparar.
- El mapa u_shape (un laberinto en forma de U) es el que expone la diferencia; en el mapa default ambos agentes se ven parecidos.

Tarea

Este ejercicio es semi-abierto: se te da el objetivo final, tú decides cómo llegar. Sigue estos pasos EN ORDEN — el diagnóstico pierde valor si lees el código antes de medir:

1. Corre ambos agentes 10 episodios cada uno en el mapa u_shape (sin --render, para que sea rápido) y anota la tasa de éxito de cada uno.
2. Antes de abrir controllers.py: escribe una hipótesis de por qué el agente "basado en modelo" — que en teoría debería ser una arquitectura más sofisticada — podría tener una tasa de éxito menor (o igual) que el reflejo simple.
3. Ahora sí, abre controllers.py y busca la clase HeuristicCalibrationController (es la que envuelve ModelBasedReflexAgent). Encuentra la línea que decide el "lado preferido" de seguimiento de pared. ¿Qué información necesita esa línea para decidir? ¿Esa información existe de verdad en nuestro entorno de un solo robot, o el código asume algo que aquí no se cumple?
4. Propón, por escrito, al menos DOS formas distintas de arreglar el problema que encuentraste, sin modificar controllers.py directamente (ese archivo pertenece al simulador original, no al material del curso).
5. Opcional / puntos extra: implementa una de tus propuestas en una copia de model_based_reflex_agent.py y vuelve a medir la tasa de éxito en u_shape. ¿Mejoró?

Entregable

Un reporte corto con: la tabla de resultados del paso 1, tu hipótesis del paso 2 (aunque haya resultado incorrecta — eso también es información valiosa), el diagnóstico del paso 3 citando la línea de código exacta, y al menos dos propuestas del paso 4.

Lista de autoevaluación

- ¿Medí ANTES de leer el código del controlador, en vez de racionalizar después de verlo?
- ¿Mi diagnóstico cita la línea de código exacta, no solo "el controlador está mal"?
- ¿Distingo entre "la variable siempre vale 0" (lo que realmente pasa) y "hay una señal contaminando el comportamiento" (lo que parece a primera vista pero no es lo que ocurre)?

- ¿Mis dos propuestas son realmente distintas entre sí, no la misma idea con otras palabras?

Manual del profesor — Actividad 2

Esta sección es para quien imparte o califica el curso; no se comparte con el alumnado antes de la entrega.

Respuesta esperada

Tasas de éxito ANTES de cualquier arreglo, en u_shape (10 episodios cada uno):

Agente	Éxitos / 10
SimpleReflexAgent (Braitenberg)	~4/10
ModelBasedReflexAgent (sin arreglo)	0/10

La línea de código exacta (en controllers.py, dentro de HeuristicCalibrationController.act()):

```
soc = obs[10] if len(obs) > 10 else 0.0
preferred = +1 if soc > 0 else -1
```

Nuestro percept de un solo robot tiene longitud 10 (índices 0-9), así que `len(obs) > 10` es siempre falso, `soc` siempre vale 0.0, y `preferred` cae siempre en -1. El controlador fue diseñado para el escenario de dos robots, donde `obs[10]` trae una señal del otro robot que decide qué lado de pared seguir.

Trampa pedagógica a vigilar

- Algunos equipos van a escribir "la señal social está contaminando el comportamiento" — es lo contrario: no hay contaminación, hay AUSENCIA total de esa señal, y el código cae en un valor por defecto fijo. Vale la pena corregir esta distinción explícitamente: "código que recibe una entrada inexistente" y "código contaminado por una entrada indebida" son diagnósticos diferentes, y solo el primero es correcto aquí.
- Algunos equipos van a proponer "borrar esa lógica del controlador" como única solución — es válida pero pierde toda la ventaja de tener un modelo interno de qué pared seguir. Empújalos a proponer algo que SÍ aproveche información real disponible en el percept.

Propuestas de arreglo válidas (para comparar con las de los alumnos)

1. Basar el lado preferido en el rumbo a la comida (`percept[8]`, `rel_angle`) — si la comida está de un lado, seguir la pared de ese lado. Es la que se implementó en este curso.
2. Sortear el lado preferido aleatoriamente al inicio de cada episodio y mantenerlo fijo durante el episodio — rompe el sesgo fijo sin usar información de la meta.
3. Alternar el lado preferido cada vez que el agente falla en reacoplar la pared varias veces seguidas (similar a como BraitenbergController ya alterna su lado de escape tras atascos repetidos).

Resultado real después de aplicar la propuesta 1 (para comparar)

Mapa	Antes del arreglo	Después del arreglo
u_shape (10 episodios)	0/10	10/10
default (10 episodios)	~4/10	7/10

Rúbrica de evaluación (10 pts)

Criterio	Puntos
Midió la tasa de éxito de ambos agentes ANTES de leer el código del controlador	2

Formuló una hipótesis propia en el paso 2 (independientemente de si acertó)	1
Diagnóstico cita la línea de código exacta y explica correctamente qué información falta	3
Distingue "ausencia de señal" de "contaminación por señal indebida"	2
Propone al menos dos soluciones genuinamente distintas entre sí	2

Variante si sobra tiempo

Pide a los equipos que implementen la propuesta 2 o 3 (en vez de la 1, ya resuelta en el curso) en su propia copia del agente, midan la tasa de éxito, y comparen las tres estrategias entre sí. Buena discusión de cierre: ¿cuál de las tres depende de menos supuestos sobre el entorno? (La 2 es la más "agnóstica" — no necesita saber nada de la meta ni de fallos previos — pero también la que menos aprovecha información disponible.)

Actividad 3: CSP — Backtracking vs. Generate-and-Test

Antes de empezar: dos términos que vas a necesitar

- Generate-and-test ("genera y prueba"): es la estrategia que TU PROPIO simulador ya usa para decidir dónde aparecen el robot y la comida. Mira la función `_sample_pose()` en `foraging_env.py`: tira un punto al azar, revisa si cumple las reglas (no estar dentro de un obstáculo, estar a la distancia correcta de la comida), y si no cumple lo descarta y prueba otro — así hasta 1000 veces. No aprende nada de un intento fallido, solo vuelve a tirar los dados.
- CSP (Constraint Satisfaction Problem / Problema de Satisfacción de Restricciones): en vez de tirar dados, se define el problema con variables (¿dónde va el robot? ¿dónde va la comida?), un dominio de valores posibles para cada una, y restricciones que deben cumplirse. La solución es UNA asignación válida — como un tablero de n-reinas ya resuelto. No hay recorrido, no hay movimiento: el problema termina en el instante en que se encuentra (o se descarta) una asignación.

Objetivo de aprendizaje

Entender que la ventaja de un método sistemático (backtracking) sobre uno ciego (generate-and-test) no siempre es "encuentra la respuesta más rápido" — a veces es algo más fundamental: la capacidad de probar con certeza que NO existe ninguna solución, algo que un método basado en azar nunca puede afirmar, sin importar cuántas veces lo intente.

Contexto

Archivos que vas a necesitar (carpeta `training_wheels/`):

- `csp_spawn.py` — implementa ambos métodos (backtracking y generate-and-test) sobre el mismo problema: dónde colocar al robot y la comida.
- `worlds/csp_dense.json` — el mapa denso diseñado para este ejercicio (cópialo a tu carpeta `worlds/real`, junto a `u_shape.json`).

Tarea

Este ejercicio es semi-abierto: se te da el objetivo, tú decides cómo explorarlo. Sigue estos pasos:

1. Corre el caso normal (`python3 csp_spawn.py --map csp_dense --seed 5`) y anota cuántos candidatos hay en cada dominio (robot y comida) y cuántos intentos/nodos necesitó cada método.
2. Corre el caso imposible (agrega `--dist_min 420 --dist_max 500 --max_attempts 5000`) y compara la salida de ambos métodos con atención — no solo los números, sino LO QUE CADA UNO PUEDE AFIRMAR al terminar.
3. En tus propias palabras (sin copiar la salida del programa): ¿por qué backtracking puede decir "no existe solución" con certeza, y generate-and-test no puede decir eso ni después de un millón de intentos?
4. Calcula tú mismo/a un rango de distancia que sea imposible para este mapa (distinto al 420-500 que ya viene de ejemplo) y confírmalo corriendo el programa con tus propios valores. Pista: el programa te dice cuántos candidatos hay en cada dominio; tú puedes calcular la distancia máxima posible entre cualquier par mirando las coordenadas que imprime `grid_world.py`.

Entregable

Un reporte corto con: los números del paso 1, la comparación del paso 2 (copiando la salida tal cual), tu explicación del paso 3, y el rango de distancia que calculaste y confirmaste en el paso 4.

Lista de autoevaluación

- ¿Puedo explicar qué es generate-and-test con mis propias palabras, sin usar la palabra "generate-and-test"?
- ¿Mi explicación del paso 3 menciona la diferencia entre "revisó TODAS las posibilidades" y "tiró dados varias veces"?
- ¿El rango de distancia que propuse en el paso 4 de verdad no tiene solución, y lo confirmé corriendo el programa (no solo calculándolo a mano)?

Manual del profesor — Actividad 3

Esta sección es para quien imparte o califica el curso; no se comparte con el alumnado antes de la entrega.

Respuesta esperada

Con el mapa `csp_dense` y semilla 5: 8 candidatos de robot, 6 de comida (48 pares posibles). Caso normal (100-300px): ambos métodos encuentran solución casi de inmediato (backtracking en 1 par revisado, generate-and-test en ~2 intentos) — el dominio es tan chico que ninguno de los dos hace "trabajo" real. Caso imposible (420-500px): backtracking revisa los 48 pares y concluye con certeza que no hay solución; generate-and-test agota 5000 intentos sin encontrar nada, pero NO puede concluir que no existe (por diseño, un método basado en azar nunca puede probar una negativa).

Trampa pedagógica a vigilar

- Muchos van a pensar que la lección es "backtracking es más rápido" — CORRÍGELO explícitamente. En el caso normal, los dos métodos son prácticamente iguales de rápidos; la ventaja real de backtracking en este ejercicio NO es velocidad, es la garantía de poder decir "no hay solución" con certeza. Si un equipo solo reporta velocidad, pídeles que vuelvan al caso imposible y expliquen esa diferencia específicamente.
- Si sobra tiempo y curiosidad: cuéntales que la primera versión de este ejercicio SÍ intentó medir velocidad, con un dominio mucho más fino (500+ candidatos) — y backtracking perdió por mucho (538 pasos vs. 14). La razón: revisar un dominio grande cuesta proporcional a su tamaño, mientras que tirar dados cuesta proporcional a $1/(\text{qué tan común es una solución})$. Backtracking le gana al azar solo cuando el dominio es CHICO comparado con qué tan raras son las soluciones — no simplemente "cuando el problema es difícil". Esta es una lección más avanzada, opcional, para grupos que ya dominen lo básico.

Rúbrica de evaluación (10 pts)

Criterio	Puntos
Corrió ambos casos (normal e imposible) y reportó los números correctamente	2
Explica generate-and-test correctamente sin usar el término (en sus propias palabras)	2
La explicación del paso 3 distingue completitud (revisar todo) de suerte (tirar dados)	3
Calculó y confirmó su propio rango de distancia imposible	3

Variante si sobra tiempo

Pregúntales: "¿por qué este problema (dónde va el robot y la comida) no muestra una diferencia de velocidad tan dramática como el n-reinas?" Respuesta esperada: aquí solo hay 2 variables con una restricción entre ellas; n-reinas tiene n variables, cada una restringida contra TODAS las demás (fila, columna, dos diagonales), lo que genera un árbol de búsqueda que crece exponencialmente — ahí es donde podar ramas completas por adelantado hace una diferencia enorme. Este ejercicio y n-reinas enseñan dos lecciones distintas y complementarias sobre CSP, no la misma lección con distinto disfraz.

Actividad 4: Minimax y Poda Alfa-Beta

Objetivo de aprendizaje

Ver, con números medidos (no solo en teoría), que la poda alfa-beta da EXACTAMENTE la misma respuesta que minimax puro, pero explorando muchos menos nodos — y que esa ventaja crece mientras más profundo se busca. Entender también que este módulo cambia una suposición importante respecto a todo lo demás del curso: aquí sí se asume información perfecta (ambos agentes conocen la posición exacta del otro).

Contexto

Archivos que vas a necesitar (carpeta `training_wheels/`):

- `minimax_search.py` — implementa minimax puro y alfa-beta sobre un juego de persecución (depredador-presa) en el mismo grafo que ya usamos para búsqueda.
- `visualize_minimax.py` — genera una imagen mostrando la persecución completa.
- El depredador quiere MINIMIZAR la distancia a la presa; la presa quiere MAXIMIZARLA. Es un juego de suma cero sobre una sola cantidad — por eso encaja con minimax, a diferencia del agente basado en utilidad (que pondera varios objetivos a la vez, no una competencia de dos jugadores).
- Hay dos formas de calificar una posición en este módulo. La primera (la que usa `minimax_search.py` para medir poda) usa la distancia real entre depredador y presa. La segunda, más clásica, usa `+1 / 0 / -1`, con un umbral de "distancia segura": `+1` si el depredador atrapa a la presa, `0` si el juego se queda en la zona de peligro (dentro de esa distancia segura) sin que haya captura — y si esa situación se repite exactamente, es un empate demostrable, no "se acabó la paciencia" — y `-1` si la presa llega a esa distancia segura sin ser atrapada. Los tres son resultados REALES del juego, medidos, no una manera de decir "no sabemos".

Tarea

Este ejercicio es semi-abierto: se te da el objetivo, tú decides cómo explorarlo.

1. Corre `python3 minimax_search.py --map u_shape --depth 4` y anota: la jugada elegida, el valor, y cuántos nodos exploró cada método (minimax puro vs. alfa-beta).
2. Repite con `--depth 2, 3, 5 y 6`. Arma una tabla con los 5 resultados (nodos de cada método + porcentaje de ahorro). ¿La ventaja de alfa-beta crece, decrece, o se mantiene igual conforme aumenta la profundidad? ¿Por qué crees que pasa eso?
3. Corre `python3 visualize_minimax.py --map u_shape --depth 3` y observa la imagen generada. ¿Cuántas jugadas le tomó al depredador atrapar a la presa?
4. Ahora corre lo mismo con `--depth 4 --rounds 20`. Compara el resultado con el paso 3 — no asumas que "más profundidad = atrapa más rápido", corrobóralo o refútalo con lo que veas.
5. Corre `python3 visualize_minimax.py --map u_shape --depth 3 --predator n0 --prey n11` (usa la versión de `+1/0/-1`). Mira la leyenda de la imagen: ¿cuál de los tres casos ocurrió? Explica, en tus propias palabras, por qué ese resultado se puede afirmar con certeza — los tres casos (`+1, 0, -1`) son ahora hechos medidos, no adivinanzas.
6. El caso `-1` (la presa escapa) es más difícil de producir que los otros dos. Prueba con `python3 visualize_minimax.py --map u_shape --depth 3 --predator n0 --prey n11 --rounds 2 --safe_distance 300` y compara contra el paso 5 (mismas posiciones, solo cambian `--rounds` y `--safe_distance`). ¿Por qué crees que hizo falta ese cambio para lograrlo?

Entregable

Un reporte con: la tabla del paso 2 (5 profundidades), tu explicación de por qué cambia el porcentaje de ahorro, la comparación de las dos imágenes de los pasos 3 y 4, y tu explicación del paso 5 (cuál de los tres casos $+1/0/-1$ te tocó y por qué se puede o no afirmar con certeza).

Lista de autoevaluación

- ¿Confirmé que minimax puro y alfa-beta SIEMPRE dan la misma jugada y el mismo valor, en las 5 profundidades que probé?
- ¿Mi tabla incluye el número exacto de nodos de ambos métodos, no solo el porcentaje?
- ¿Evité asumir que más profundidad siempre ayuda al depredador, y en vez de eso lo comprobé corriendo el programa?
- ¿Distingo correctamente "0: empate demostrado por un ciclo real" de "-1: todavía no sabemos", y no los trato como si fueran lo mismo?

Manual del profesor — Actividad 4

Esta sección es para quien imparte o califica el curso; no se comparte con el alumnado antes de la entrega.

Respuesta esperada

Profundidad	Minimax puro (nodos)	Alfa-beta (nodos)	Ahorro
2	12	12	0%
3	139	47	66.2%
4	1,379	297	78.5%
5	11,997	718	94.0%
6	116,321	2,863	97.5%

La misma jugada y el mismo valor en las 5 profundidades — la poda nunca cambia la respuesta, solo el trabajo para llegar a ella. El ahorro CRECE con la profundidad (0% a profundidad 2, hasta 97.5% a profundidad 6): a mayor profundidad, más ramas hay para podar. A profundidad 2 el árbol es tan chico que no hay margen para podar nada — vale la pena mencionarlo si algún equipo espera que la poda "siempre ahorre algo".

Trampa pedagógica a vigilar

- El paso 4 tiene una trampa a propósito: con las mismas posiciones iniciales, profundidad 3 SÍ atrapa a la presa (5 jugadas) pero profundidad 4 NO la atrapa nunca — cae en un ciclo infinito de evasión (se puede confirmar imprimiendo la trayectoria completa: se repite exactamente). La razón: es autojuego — depredador y presa usan la MISMA profundidad, así que darle más anticipación al depredador se la da igual de gratis a la presa. Más profundidad no favorece automáticamente a ninguno de los dos lados; depende de la topología específica del grafo en esa profundidad exacta.
- Si algún equipo concluye "entonces la profundidad no importa", corrígelo: sí importa (cambia completamente el resultado), simplemente no importa de forma monótona o predecible cuando ambos jugadores son igual de "listos".
- El caso -1 SÍ es una victoria real de la presa (llegó a la distancia segura, es un hecho medido) — esto es distinto de una versión anterior de este ejercicio, donde -1 significaba "se acabaron las rondas sin saber qué iba a pasar", una confesión de incertidumbre disfrazada de resultado. Si tienes ese material viejo, corrígelo: el error real era que la evaluación al cortar la búsqueda no medía nada, solo devolvía -1 a ciegas. La corrección fue agregar un umbral de distancia segura (SAFE_DISTANCE) y medir la distancia real contra ese umbral en cada corte de búsqueda.
- En el paso 6, la lección importante: la presa NO le gana al depredador por jugar mejor — le gana porque el depredador se quedó sin turnos suficientes para alcanzarla. Barriendo todos los pares de nodos posibles en u_shape con profundidad 3, el caso -1 NUNCA ocurrió de forma natural con 20 rondas — solo aparece si se acorta mucho el presupuesto de rondas. Vale la pena discutir si eso cuenta como "la presa gana" de verdad, o si depende de qué tan generoso sea el número de rondas permitidas — no hay una respuesta única correcta, es la discusión en sí lo que importa.

Rúbrica de evaluación (10 pts)

Criterio	Puntos
Tabla completa y correcta de las 5 profundidades	2
Confirma que minimax y alfa-beta siempre coinciden en jugada y valor	1
Explica correctamente por qué el ahorro crece con la profundidad	2
Detecta y explica el hallazgo del ciclo de evasión en el paso 4	2

Explica correctamente su resultado +1/0/-1 del paso 5, distinguiendo certeza de incertidumbre	3
---	---

Variante si sobra tiempo

Pídeles que prueben otro mapa (`--map n_shape` o `--map default`) y repitan la tabla de profundidades. ¿El patrón de crecimiento del ahorro se mantiene parecido, o cambia mucho con la forma del grafo? Buena forma de que noten que el ahorro de la poda depende del orden en que se exploran las ramas (por eso el código ordena los movimientos con `sorted()` — un orden distinto podría podar más o menos).

El ejemplo -1 del paso 6 (con `--rounds 2`) es el mínimo indispensable para demostrar el caso, pero se ve poco dinámico (un salto cada uno y se acaba). Si quieres un ejemplo más vistoso para mostrar en clase, corre `python3 visualize_minimax.py --map u_shape --depth 3 --predator n10 --prey n0 --rounds 6 --safe_distance 200` — da 6 jugadas de persecución real antes de que se acabe el presupuesto de rondas, incluyendo un tramo donde la presa se mete al hueco de la U y sale de nuevo.

Actividad 5: Q-Learning — Aprender sin Conocer el Mapa

Objetivo de aprendizaje

Entender la diferencia real entre planear (A^* , que ya conoce el mapa completo desde el inicio) y aprender por refuerzo (Q-learning, que NO conoce qué movimientos son válidos y solo lo descubre probando). Ver la curva de aprendizaje clásica: el desempeño mejora con la experiencia, no de golpe.

Contexto

Archivos que vas a necesitar (carpeta training_wheels/):

- qlearning.py — entrena una tabla Q sobre el mismo grafo grueso que ya usamos para búsqueda y minimax.
- visualize_qlearning.py — dibuja la política aprendida como flechas sobre el grafo.
- La diferencia clave con A^* : el agente NUNCA recibe la lista de aristas del grafo. Su espacio de acciones es "intenta moverte al nodo X", para CUALQUIER nodo X del grafo — si (nodo_actual, X) no es una arista real, el intento simplemente falla (el agente se queda donde estaba y recibe un castigo), como chocar contra una pared que no sabía que estaba ahí. Solo prueba y error, repetido muchas veces, le enseña cuáles intentos sí funcionan.
- Recompensas: -1 por cada paso (para que aprenda rutas eficientes), -5 si el movimiento intentado no era una arista real, +100 al llegar a la comida.

Tarea

Este ejercicio es semi-abierto: se te da el objetivo, tú decides cómo explorarlo.

1. Corre `python3 qlearning.py --map u_shape --episodes 2000` y anota: el promedio de pasos por episodio en los primeros 10 episodios vs. los últimos 10, y cuántos de los estados aprendieron un movimiento correcto.
2. Corre `python3 visualize_qlearning.py --map u_shape --episodes 2000` y observa la imagen: ¿todas las flechas son verdes? ¿Hacia dónde apuntan en general (nota algo sobre su dirección respecto a la comida)?
3. Ahora corre lo mismo pero con `--episodes 10` (muy pocos). Compara la imagen contra la del paso 2. ¿Cuántas flechas rojas aparecen? Elige una y explica, en tus propias palabras, por qué esa flecha específica quedó mal — no en general, sino para ESE nodo en particular.
4. Prueba con `--episodes 20, 30, 50 y 100`. ¿En qué número de episodios, aproximadamente, todas las flechas se vuelven verdes en tu corrida? (Nota: el resultado exacto puede variar un poco entre corridas por la exploración aleatoria — corre cada uno un par de veces si el número te sorprende.)
5. Explica, en un párrafo, por qué NO le dimos al agente la lista de aristas del grafo desde el principio (como sí hicimos con A^*). ¿Qué estaríamos demostrando si sí se la hubiéramos dado?

Entregable

Un reporte con: los números del paso 1, tu observación del paso 2, la explicación de una flecha roja específica del paso 3, el número de episodios donde convergió en el paso 4, y tu explicación del paso 5.

Lista de autoevaluación

- ¿Reporté números concretos (promedio de pasos, cantidad de flechas correctas), no solo "mejoró"?

- ¿Mi explicación de la flecha roja es específica de ESE nodo, no una explicación genérica de "le faltó entrenamiento"?
- ¿Mi respuesta al paso 5 explica la diferencia entre planear y aprender, no solo repite la definición del Contexto?

Manual del profesor — Actividad 5

Esta sección es para quien imparte o califica el curso; no se comparte con el alumnado antes de la entrega.

Respuesta esperada

Con `u_shape` y 2000 episodios (corrida de referencia): promedio de pasos baja de ~99.6 (primeros 10 episodios) a ~2.8 (últimos 10). 22/22 estados aprenden un movimiento correcto. Con solo 10 episodios, típicamente 17/22 correctos (5 flechas rojas) — el número exacto varía por la semilla aleatoria, no busques que coincida exacto con esto, lo importante es que HAYA flechas rojas visibles.

Trampa pedagógica a vigilar

- En el paso 3, algunos van a explicar la flecha roja con una frase genérica ("porque no entrenó lo suficiente"). Empújalos a ser específicos: ese nodo en particular casi no fue visitado durante los pocos episodios de exploración aleatoria (el agente empieza en un nodo ALEATORIO cada episodio, así que con pocos episodios, algunos nodos simplemente nunca les tocó como punto de partida ni fueron visitados de paso).
- En el paso 5, la respuesta débil es "para que aprenda solo" sin explicar qué se pierde al dárselo. La respuesta fuerte: si le diéramos las aristas de entrada, Q-learning terminaría redescubriendo el mismo camino que A* ya calcula al instante, solo que mucho más lento — no demostraría nada que A* no demuestre ya mejor. Aprender A CIEGAS es lo que separa conceptualmente a un agente que aprende de uno que planea.

Rúbrica de evaluación (10 pts)

Criterio	Puntos
Reporta correctamente los números de entrenamiento del paso 1	2
Observación correcta de la imagen bien entrenada (paso 2)	1
Explicación específica (no genérica) de una flecha roja concreta	3
Identifica un número de episodios razonable de convergencia (paso 4)	1
Explica correctamente por qué aprender a ciegas es el punto central del ejercicio (paso 5)	3

Variante si sobra tiempo

La Fase 2 ya existe: `python3 demo_qlearning_live.py --map u_shape --episodes 2000` entrena la misma tabla y la usa para mover al robot de verdad en el simulador, guardando un GIF del recorrido completo. Es la mejor forma de cerrar la actividad en clase — después de ver flechas y números abstractos, verlo funcionar sobre el robot real conecta todo. Menciona que esto necesitó la misma excepción de diseño que se había dejado pendiente para `goal_based_agent.py`: leer la posición real del robot directamente del entorno, porque el `percept` normal no la incluye.

Actividad 6: GA vs. Q-Learning — Dos Formas de Aprender

Objetivo de aprendizaje

Comparar, con números medidos (no solo en teoría), dos paradigmas de aprendizaje que resuelven EXACTAMENTE el mismo problema: Q-learning busca en el espacio de valores (una tabla que se actualiza experiencia por experiencia); el algoritmo genético busca en el espacio de políticas completas (una población de tablas completas compitiendo entre sí). Ninguno es "mejor" en abstracto — la tarea es entender sus diferencias reales, medidas.

Contexto

Archivos que vas a necesitar (carpeta training_wheels/):

- `qlearning.py` y `ga_learning.py` — ya los conoces de las actividades anteriores; ambos ahora imprimen también un conteo de interacciones con el entorno, que vas a necesitar aquí.
- `visualize_ga_population.py` — la única imagen que muestra algo que Q-learning estructuralmente no puede mostrar: una POBLACIÓN completa de soluciones compitiendo, no un solo agente aprendiendo.
- `demo_qllearning_live.py` y `demo_ga_live.py` — ambos mueven al robot real con la política ya entrenada; internamente comparten el mismo código de dirección (`action_toward`), así que cualquier diferencia visual entre los dos GIFs viene de la política aprendida, no de cómo se mueve el robot.

Tarea

Este ejercicio es semi-abierto: se te da el objetivo, tú decides cómo explorarlo.

1. Corre `python3 qllearning.py --map u_shape --episodes 2000` y anota el total de interacciones reales que imprime al final. Corre `python3 ga_learning.py --map u_shape --generations 100` y anota el presupuesto de interacciones en el PEOR CASO que imprime. ¿Son comparables estos dos números directamente? ¿Por qué sí o por qué no?
2. Corre `python3 visualize_ga_population.py --map u_shape --generations 100` y observa la imagen. Describe, en tus propias palabras, qué muestra esta imagen que ninguna imagen de Q-learning podría mostrar jamás, sin importar cuántos episodios entrenaras.
3. Corre `demo_qllearning_live.py` y `demo_ga_live.py` en el mismo mapa (usa `--spawn_seed` para que ambos arranquen del mismo punto si quieres una comparación justa). Compara cuántos pasos reales necesitó cada uno para llegar a la comida.
4. Revisa la tabla Q (`python3 qllearning.py`) y busca si hay empates entre las mejores acciones de un mismo estado (marcados con el mismo valor). Ahora revisa la política del GA (puedes imprimir `best_chromosome` desde `ga_learning.py` en una consola de Python) ¿La política del GA también tiene esa propiedad de "empate", o se ve distinta? No asumas la respuesta, corre el código.
5. Escribe un párrafo de cierre: si tuvieras un grafo mucho más grande (digamos, 200 nodos en vez de 22), ¿cuál de los dos métodos esperarías que escale peor, y por qué? (No hace falta que lo pruebes — esto es una predicción justificada, no una medición.)

Entregable

Un reporte con: los dos números del paso 1 y tu explicación de si son comparables, tu descripción del paso 2, la comparación de pasos reales del paso 3, tu hallazgo sobre empates del paso 4, y tu predicción justificada del paso 5.

Lista de autoevaluación

- ¿Explicué por qué comparar "interacciones reales" de Q-learning contra el "peor caso" del GA no es una comparación perfectamente justa, en vez de solo poner los dos números uno al lado del otro?
- ¿Mi descripción del paso 2 menciona específicamente la idea de POBLACIÓN, no solo "se ve diferente"?
- ¿Mi predicción del paso 5 tiene una razón concreta detrás (tamaño del cromosoma, tamaño de la tabla, etc.), no solo una corazonada?

Manual del profesor — Actividad 6

Esta sección es para quien imparte o califica el curso; no se comparte con el alumnado antes de la entrega.

Respuesta esperada

Corrida de referencia en u_shape: Q-learning usó 23,787 interacciones reales en 2000 episodios. El GA tiene un presupuesto de PEOR CASO de 3,168,000 ($\text{población} = 24 \times \text{generaciones} = 100 \times 22 \text{ estados} \times 60 \text{ pasos máximo}$), pero el uso REAL es mucho menor porque la mayoría de los rollouts terminan mucho antes de agotar los 60 pasos en cuanto un cromosoma es medianamente bueno — el código actual no instrumenta el conteo exacto del GA, así que la comparación directa de estos dos números tal cual NO es justa (uno es real, el otro es una cota superior).

Trampa pedagógica a vigilar

- La trampa principal del paso 1 es que algunos equipos van a comparar 23,787 contra 3,168,000 y concluir de inmediato "Q-learning es mucho más eficiente" — CORRÍGELO. Esa conclusión compara un número REAL contra una COTA SUPERIOR, no dos cantidades del mismo tipo. Sin instrumentar el conteo real del GA (que el código actual no hace), no se puede afirmar cuál usa menos interacciones de verdad — la respuesta honesta al paso 1 es "no son comparables tal cual", no un número ganador.
- En el paso 2, la respuesta débil es "se ven varios mapas". La respuesta fuerte menciona explícitamente que son múltiples INDIVIDUOS/SOLUCIONES coexistiendo y compitiendo al mismo tiempo — Q-learning, en cualquier punto de su entrenamiento, solo tiene UNA tabla, nunca varias compitiendo entre sí.

Rúbrica de evaluación (10 pts)

Criterio	Puntos
Identifica correctamente que los dos números del paso 1 no son comparables tal cual	3
Describe el paso 2 mencionando explícitamente el concepto de población	2
Reporta correctamente la comparación de pasos reales del paso 3	2
Predicción del paso 5 con una razón concreta, no una corazonada	3

Variante si sobra tiempo

Para un grupo avanzado: pídeles que instrumenten ellos mismos rollout_fitness en ga_learning.py para contar las interacciones reales (no el peor caso), y que rehagan la comparación del paso 1 con números genuinamente comparables. Es un buen ejercicio de "no confíes en un número que no mediste tú mismo", el mismo espíritu que guió la construcción de todo este curso.